Piano music continuation with Transformers

Mohammed Zahoor Mashahir - PES1UG24CS579
Mohammed Ibrahim - PES1UG24CS578
UE24CS352A Machine Learning | 13 October 2026

Our project generates a continuation of a solo-piano MIDI prompt. We use two versions of our Transformer: an initially trained model and a final model obtained after post-training. They share the same network architecture. The distinction is how their weights have been trained. A user supplies a musical starting idea, and the selected checkpoint predicts the events that follow it. The system exports the result as editable MIDI and piano audio.

Why we use a Transformer

We chose a decoder-only Transformer because music is an ordered sequence with relationships across many events. A later note can relate to an earlier chord, rhythm or melodic phrase. Self-attention lets a token use information from earlier positions within the available context. A causal mask permits only the current and previous positions, matching the task of predicting the next token without seeing the future. This makes the same network useful for learning from complete performances and continuing a supplied prompt. Music Transformer research motivates this use of self-attention [1].

Musical input representation

We use MAESTRO piano MIDI performances and keep training, validation and test splits separate. MIDI describes notes and performance timing directly, so the model learns musical events rather than audio waveforms. Each piano note uses a pitch-and-velocity token followed by onset and duration tokens. Timing uses 10 ms steps and five-second segments. Prefix and boundary tokens identify the instrument and sequence structure. This representation exposes melody, rhythm and dynamics while keeping the generated music editable.

Our decoder architecture

Component	Configuration	Purpose
Input and context	17,727 token IDs; 2,048-token context	Represent events and bound attention cost.
Embedding	512 dimensions	Turn each token ID into a learned feature vector.
Decoder stack	8 blocks; 8 heads per attention layer	Combine relationships among earlier events.
Position encoding	Rotary positions (RoPE)	Encode token order in attention [2].
Feed-forward layer	SwiGLU; width 1,536	Transform features through a learned gate [4].
Stability and regularization	Pre-LayerNorm, residual connections, dropout 0.1	Support gradient flow and regularize training [3].
Output head	Final LayerNorm; tied embedding weights	Produce next-token scores while reusing parameters.

Inside each block, normalization precedes causal multi-head attention. The block adds the attention output back to its input through a residual connection. A second normalization feeds SwiGLU, whose SiLU-activated gate controls a parallel feature projection; another residual connection adds that output. RoPE acts on queries and keys. Token order and musical time are distinct: explicit onset and duration tokens carry time. The 36.36M-parameter design balances expressive capacity with our local 12 GB GPU budget.

The initial Transformer

The initial Transformer starts with randomly initialized weights and learns next-token prediction from piano performances. Token sequences are cropped within a single piece and end on complete note groups. Inputs and targets are shifted by one position: each position predicts the token that follows it. Cross-entropy trains the predicted distribution against the real next token. Padding and the supplied instrument/start prefix do not contribute to loss. This stage builds the model's general knowledge of the event representation and the musical patterns present in the corpus.

The final Transformer after post-training

The final Transformer descends from the initial checkpoint through further training. We retain the decoder stack, vocabulary, context length and output head, then continue updating the learned weights. Post-training includes examples tailored to the continuation task. In continuation-focused examples, the model receives a short musical prompt as context and the loss scores the tokens after that prompt. We choose complete onset groups so a chord stays intact and leave enough following notes as targets. Ordinary next-token examples remain in the training mix, preserving the general prediction task.

We also augment whole passages with one consistent pitch shift and one consistent time scale. Moving all notes by the same number of semitones preserves intervals and chord relationships. Scaling onsets and durations together preserves their timing relationships up to token quantization. Shifts must keep every pitch in the piano range. These examples give the model different keys and timing scales for related musical structures. The purpose is to align training with continuing an existing idea; the architecture stays unchanged.

Optimization and checkpointing

Both stages use AdamW and gradient clipping. BF16 and gradient accumulation let us train with the available GPU memory. The final stage uses a lower learning rate so updates are smaller around the already learned solution. Checkpoints preserve the weights, optimizer and random states so interrupted training can resume consistently.

How the architecture generates music

At inference, the system tokenizes the user's prompt and passes it to either checkpoint. The output head gives scores for the next token. Temperature and min-p sampling control variation, while event constraints keep note groups valid. The chosen token joins the sequence and becomes context for the next prediction. This repeats until an end token, the requested onset horizon or the context limit stops generation. We decode the events to MIDI and use a fixed piano renderer for audio. Note releases can continue after the last generated onset. The Transformer models event structure and expression; the renderer provides the piano sound.

Contributions

Equal contribution allocation: Mohammed Zahoor Mashahir covers data preparation, tokenization and Transformer training. Mohammed Ibrahim covers generation, piano rendering, the application and submission documentation. Both share method interpretation and demonstration preparation. Development used an AI coding assistant.

References

[1] Huang et al. Music Transformer. https://arxiv.org/abs/1809.04281

[2] Su et al. RoFormer. https://arxiv.org/abs/2104.09864

[3] Xiong et al. On Layer Normalization in the Transformer Architecture. https://arxiv.org/abs/2002.04745

[4] Shazeer. GLU Variants Improve Transformer. https://arxiv.org/abs/2002.05202

Data: MAESTRO, https://magenta.tensorflow.org/datasets/maestro (CC BY-NC-SA 4.0). Implementation: src/music_project/models/networks.py; configs/transformer_full.yaml; configs/transformer_musical.yaml.